Document number	P0792R4
Date	2018-10-07
Reply-to	Vittorio Romeo <vittorio.romeo@outlook.com (mailto:vittorio.romeo@outlook.com)>
Audience	Library Working Group (LWG)
Project	ISO JTC1/SC22/WG21: Programming Language C++

`function_ref`: a non-owning reference to a `Callable`

Abstract

This paper proposes the addition of `function_ref<R(Args...)>` to the Standard Library, a *"vocabulary type"* for non-owning references to `Callable` objects.

Changelog and polls

From R3 to R4

Changes

- Stripped qualifiers from exposition-only `erased_fn_type`;
- Removed `constexpr` due to implementation concerns - this can be re-introduced in a future paper;
- Changed wording to give instructions to the editor;
- Added paragraph numbers and reordered sections;
- Added brief before-class synopsis;
- Added an explicit bullet point for trivial copyability;

- Removed defaulted functions (copy constructor and assignment) from specification;
- Reworded specification following P1369;
- Mention exposition-only members in template constructor;
- Add “see below” to `noexcept` on `operator()`.

From R2 to R3

Changes

- Removed `!f` precondition for construction/assignment from `std::function f`;
- `function_ref::operator()` is now unconditionally `const`-qualified.

Polls

Do we want to remove the precondition that `!f` must hold when `function_ref` is constructed/assigned from an instance of `std::function f`? SF F N A SA 1 8 1 0 0
Should `function_ref::operator()` be unconditionally `const`-qualified? SF F N A SA 8 2 0 1 0
Should `function_ref` fully support the `Callable` concept at the potential cost of `sizeof(function_ref) > sizeof(void(*)()) * 2`? SF F N A SA 6 4 0 0 0

From R1 to R2

Changes

- Made *copy constructor* and *copy assignment* `= default`;
- Changed uses of `std::decay_t` to `std::remove_cvref_t`;
- Added "exposition only" `void*` and *pointer to function* data members;
- Moved *"Open questions"* section to *"Annex: previously open questions"*;
- Change `function_ref(F&&)` constructor's precondition to use `remove_cvref_t` to check if `F` is an instance of the `function` class template;
- Dropped `function_ref<Signature>::` qualification in member function specification.

Polls

We want to prevent construction of std::function from std::function_ref (but not other callable-taking things like std::bind). SF F N A SA 0 0 4 8 0
We want to revise the paper to include discussion of ref-qualified callables. SF F N A SA 0 3 6 6 0
Forward paper as-is to LWG for C++20? SF F N A SA 3 9 3 0 0

From R0 to R1

Changes

- Removed empty state and comparisons with `nullptr`;
- Removed default constructor;
- Added support for `noexcept` and `const`-qualified function signatures *(these are propagated to `function_ref::operator()`)*;
- Added deduction guides for function pointers and arbitrary callable objects with well-formed `&remove_reference_t<F>::operator()`;
- Added two new bullet points to “Open questions”;
- Added “Example implementation”;
- Added “Feature test macro”;
- Removed `noexcept` from constructor and assignment.

Semantics: pointer versus reference

> option 1
>
> function_ref, non-nullable, not default constructible
>
> option 2
>
> function_ptr, nullable, default constructible
>
> We want 1 and 2
>
> SF F N A SA
>
> 1 2 8 3 6
>
> ref vs ptr
>
> SR R N P SP
>
> 6 5 2 5 0

The poll above clearly shows that the desired direction for `function_ref` is towards a *non nullable*, *non default-constructible* reference type. This revision (P0792R2) removes the “empty state” and default constructibility from the proposed `function_ref`. If those semantics are required by users, they can trivially wrap `function_ref` into an `std::optional<function_ref</* ... */>>`.

`target` and `target_type`

> We want target and target-type (consistent with std::function) if they have no overhead
>
> Unanimous consent
>
> We want target and target-type (consistent with std::function) even though they have overhead
>
> SF F N A SA
>
> 0 0 1 9 4

I am not sure whether `target` and `target_type` can be implemented without introducing overhead. I seek the guidance of the committee or any interested reader to figure that out. If they require overhead, I agree with the poll: they will be left out of the proposal.

Table of contents

Overview

Since the advent of C++11 writing more functional code has become easier: functional programming patterns and idioms have become powerful additions to the C++ developer's toolbox. **"Higher-order functions"** are one of the key ideas of the functional paradigm - in short, they are functions that take functions as arguments and/or return functions as results.

The need of referring to an existing `Callable` object comes up often when writing functional C++ code, but the Standard Library unfortunately doesn't provide a flexible facility that allows to do so. Let's consider the existing utilities:

- **Pointers to functions** are only useful when the entity they refer to is stateless *(i.e. a non-member function or a capture-less lambda)*, but they are cumbersome to use otherwise. Fully supporting the `Callable` concept requires also explicitly dealing with **pointers to member functions** and **pointers to data members**.
- **`std::function`** seamlessly works with `Callable` objects, but it's a *"general-purpose polymorphic function wrapper"* that may introduce unnecessary overhead and that **owns** the `Callable` it stores. `std::function` is a great choice when an owning type-erased wrapper is required, but it's often abused when its ownership semantics and its flexibility are not required.
  - Note that when `std::function` is constructed/assigned with a `std::reference_wrapper` to a `Callable`, it has reference semantics.
  - Another limitation of `std::function` is the fact that the stored `Callable` must be `CopyConstructible`.
- **Templates** can be used to avoid unnecessary costs and to uniformly handle any `Callable` object, but they are hard to constrain to a particular signature and force code to be defined in headers.

This paper proposes the introduction of a new `function_ref` class template, which is akin to `std::string_view`. This paper describes `function_ref` as a **non-owning lightweight wrapper** over any `Callable` object.

Motivating example

Here's one example use case that benefits from *higher-order functions*: a `retry(n, f)` function that attempts to synchronously call `f` up to `n` times until success. This example might model the real-world scenario of repeatedly querying a flaky web service.

```
struct payload { /* ... */ };

// Repeatedly invokes `action` up to `times` repetitions.
// Immediately returns if `action` returns a valid `payload`.
// Returns `std::nullopt` otherwise.
std::optional<payload> retry(std::size_t times, /* ????? */ action);
```

The passed-in `action` should be a `Callable` which takes no arguments and returns `std::optional<payload>`. Let's see how `retry` can be implemented with various techniques:

- Using *pointers to functions*:

  ```
  std::optional<payload> retry(std::size_t times,
                               std::optional<payload>(*action)())
  {
      /* ... */
  }
  ```

  ***(on godbolt.org)* (https://godbolt.org/g/UQbZYp)**

  - **Advantages**:
    - Easy to implement: no need to use a `template` or any explicit constraint *(e.g. `std::enable_if_t<...>`)*. The type of the pointer specifies exactly which functions can be passed, no extra constraints are required.
    - Minimal overhead: no allocations, no exceptions, and `action` is as big as a pointer.
      - Modern compilers are able to completely inline the call to `action`, producing optimal assembly.
  - **Drawbacks**:
    - This technique doesn't support stateful `Callable` objects.

- Using a `template`:

```
template <typename F>
auto retry(std::size_t times, F&& action)
    -> std::enable_if_t<std::is_invocable_r_v<std::optional<payload>, F&&>,
                        std::optional<payload>>
{
    /* ... */
}
```
(on godbolt.org) **(https://godbolt.org/g/AGikkz)**

- **Advantages**:
  - Supports arbitrary `Callable` objects, such as stateful closures.
  - Zero-overhead: no allocations, no exceptions, no indirections.
- **Drawbacks**:
  - Harder to implement and less readable: users must use `std::enable_if_t` and `std::invocable_r_v` to ensure that `action`'s signature is properly constrained.
  - `retry` must be defined in a header file. This might be undesiderable when trying to minimize compilation times.

- Using `std::function`:

```
std::optional<payload> retry(std::size_t times,
                             std::function<std::optional<payload>()> action)
{
    /* ... */
}
```
(on godbolt.org) **(https://godbolt.org/g/t9FH9b)**

- **Advantages**:
  - Supports arbitrary `Callable` objects, such as stateful closures.
  - Easy to implement: no need to use a `template` or any explicit constraint. The type fully constrains what can be passed.
- **Drawbacks**:
  - Unclear ownership semantics: `action` might either own the the stored `Callable`, or just refer to an existing `Callable` if initialized with a `std::reference_wrapper`.
  - Can potentially have significant overhead:

        - Even though the implementation makes use of SBO *(small buffer optimization)*, `std::function` might allocate if the stored object is large enough. This requires one extra branch on construction/assignment, one potential dynamic allocation, and makes `action` as big as the size of the internal buffer.
        - If the implementation doesn't make use of SBO, `std::function` will always allocate on construction/assignment.
        - Modern compilers are not able to inline `std::function`, often resulting in very poor assembly compared to the previously mentioned techniques.
    - Mandatory use of exceptions: `std::function` might throw if an allocation fails, and throws `std::bad_function_call` if it's invoked while unset.

- Using the proposed `function_ref`:

```
std::optional<payload> retry(std::size_t times,
                             function_ref<std::optional<payload>()> action)
{
    /* ... */
}
```

(on godbolt.org) **(https://godbolt.org/g/DvWKVH)**

    - **Advantages**:
        - Supports arbitrary `Callable` objects, such as stateful closures.
        - Easy to implement: no need to use a `template` or any constraint. The type fully constrains what can be passed.
        - Clear ownership semantics: `action` is a **non-owning** reference to an existing `Callable`.
        - Small overhead: no allocations, no exceptions, and `action` is as big as two pointers.
            - Modern compilers are able to completely inline the call to `action`, producing optimal assembly.

Impact on the Standard

This proposal is a pure library extension. It does not require changes to any existing part of the Standard.

Alternatives

The only existing viable alternative to `function_ref` currently is `std::function` + `std::reference_wrapper`. The Standard guarantees that when a `std::reference_wrapper` is used to construct/assign to a `std::function` no allocations will occur and no exceptions will be thrown.

Using `std::function` for non-owning references is suboptimal for various reasons.

1. The ownership semantics of a `std::function` are unclear - they change depending on whether or not the `std::function` was constructed/assigned with a `std::reference_wrapper`.

   ```
   void foo(std::function<void()> f);
   // `f` could be referring to an existing Callable, or could own one.

   void bar(function_ref<void()> f);
   // `f` unambiguously is a non-owning reference to an existing Callable.
   ```

2. This technique doesn't work with temporaries. This is a huge drawback as it prevents stateful temporary lambdas from being passed as callbacks.

   ```
   void foo(std::function<void()> f);

   int main()
   {
       int x = 0;
       foo(std::ref([&x]{ ++x; })); // does not compile
   }
   ```

   ***(on godbolt.org)* (https://godbolt.org/g/DPQ7ku)**

   The code above doesn't compile, as `std::ref` only accepts non-`const` lvalue references *(additionally, `std::cref` is explicitly deleted for rvalue references)*. Avoiding the use of `std::ref` breaks the guarantee that `f` won't allocate or throw an exception on construction.

3. `std::function` is harder for compilers to optimize compared to the proposed `function_ref`. This is true due to various reasons:

   - `std::function` can allocate and/or throw exceptions on construction and/or assigment.
   - `std::function` might use SBO, which could require an additional branch during construction/assignment, make inlining more difficult, and unnecessarily increase memory usage.

   Rough benchmarks comparing the generated assembly of a *`std::function` parameter* and a *`function_ref` parameter* against a *template parameter* show that:

- `std::function`, on average, generates approximately 5x more assembly than a template parameter.
- `function_ref`, on average, generates approximately 1.5x more assembly than a template parameter.

A description of the benchmarking techniques used and the full results can be found on my article *"passing functions to functions"* [1].

Changes to `<functional>` header

Add the following to `[functional.syn]`:

```
namespace std
{
    // ...

    template <typename Signature> class function_ref;

    template <typename Signature>
    void swap(function_ref<Signature>& lhs, function_ref<Signature>& rhs) noexcept;

    // ...
}
```

Class synopsis

Create a new section "Class template `function_ref`", `[functionref]`" with the following:

```
namespace std
{
    template <typename Signature>
    class function_ref
    {
        void* erased_object; // exposition only

        R(*erased_function)(Args...); // exposition only
        // `R`, and `Args...` are the return type, and the parameter-type-list,
        // of the function type `Signature`, respectively.

    public:
        function_ref(const function_ref&) noexcept = default;

        template <typename F>
        function_ref(F&&);

        function_ref& operator=(const function_ref&) noexcept = default;

        template <typename F>
        function_ref& operator=(F&&);

        void swap(function_ref&) noexcept;

        R operator()(Args...) const noexcept(see below);
        // `R` and `Args...` are the return type and the parameter-type-list
        // of the function type `Signature`, respectively.
    };

    template <typename Signature>
    void swap(function_ref<Signature>&, function_ref<Signature>&) noexcept;

    template <typename R, typename... Args>
    function_ref(R (*)(Args...)) -> function_ref<R(Args...)>;

    template <typename R, typename... Args>
    function_ref(R (*)(Args...) noexcept) -> function_ref<R(Args...) noexcept>;

    template <typename F>
    function_ref(F) -> function_ref<see below>;
}
```

1. `function_ref<Signature>` is a `Cpp17CopyConstructible` and `Cpp17CopyAssignable` reference to an `Invocable` object with signature `Signature`.
2. `function_ref<Signature>` is a trivially copyable type.
3. The template argument `Signature` shall be a non-`volatile`-qualified function type.

Specification

```
template <typename F>
function_ref(F&& f);
```

- *Constraints:* `is_same_v<remove_cvref_t<F>, function_ref>` is `false` and
  - If `Signature` has a `noexcept` specifier: `is_nothrow_invocable_r_v<R, cv-qualifiers F&, Args...>` is `true`;
  - Otherwise: `is_invocable_r_v<R, cv-qualifiers F&, Args...>` is `true`.

  Where `R`, `Args...`, and `cv-qualifiers` are the *return type*, the *parameter-type-list*, and the sequence "*cv-qualifier-seq-opt*" of the function type `Signature`, respectively.
- *Expects:* `f` is neither a null function pointer value nor a null member pointer value.
- *Effects:* Constructs a `function_ref` referring to `f`.
- *Remarks:* `erased_object` will point to `f`. `erased_function` will point to a function whose invocation is equivalent to `return INVOKE<R>(f, std::forward<Args>(xs)...);`, where `f` is qualified with the same *cv-qualifiers* as the function type `Signature`.

```
template <typename F>
function_ref& operator=(F&&);
```

- *Constraints:* `is_same_v<remove_cvref_t<F>, function_ref>` is `false` and
  - If `Signature` has a `noexcept` specifier: `is_nothrow_invocable_r_v<R, cv-qualifiers F&, Args...>` is `true`;
  - Otherwise: `is_invocable_r_v<R, cv-qualifiers F&, Args...>` is `true`.

Where `R`, `Args...`, and `cv-qualifiers` are the *return type*, the *parameter-type-list*, and the sequence "*cv-qualifier-seq-opt*" of the function type `Signature`, respectively.

- *Expects:* `f` is neither a null function pointer value nor a null member pointer value.
- *Ensures:* `*this` refers to `f`.
- *Returns:* `*this`.

```
void swap(function_ref& rhs) noexcept;
```

- *Effects:* Exchanges the values of `*this` and `rhs`.

```
R operator()(Args... xs) qualifiers noexcept(see below);
```

- *Effects:* Equivalent to `return INVOKE<R>(f, std::forward<Args>(xs)...);`, where `f` is the callable object referred to by `*this`, qualified with the same *cv-qualifiers* as the function type `Signature`.
- *Remarks:* `R` and `Args...` are the return type and the parameter-type-list of the function type `Signature`, respectively. The expression inside `noexcept` is the sequence "noexcept-specifier-opt" of the function type `Signature`.

```
template <typename F>
function_ref(F) -> function_ref<see below>;
```

- *Constraints:* `&F::operator()` is well-formed when treated as an unevaluated operand.
- *Remarks:* If `decltype(&F::operator())` is of the form `R(G::*)(A...) qualifiers` for a class type `G`, then the deduced type is `function_ref<R(A...) qualifiers>`.

```
template <typename Signature>
void swap(function_ref<Signature>& lhs, function_ref<Signature>& rhs) noexcept;
```

- *Effects:* Equivalent to `lhs.swap(rhs)`.

Feature test macro

Append to §17.3.1 General `[support.limits.general]`'s Table 36 one additional entry:

Macro name	Value	Headers
__cpp_lib_function_ref	201811L	`<functional>`

Example implementation

The most up-to-date implementation, created by Simon Brand, is available on GitHub/TartanLlama/function_ref (https://github.com/TartanLlama/function_ref).

An older example implementation is available here on GitHub/SuperV1234/Experiments (https://github.com/SuperV1234/Experiments/blob/master/function_ref.cpp).

Existing practice

Many facilities similar to `function_ref` exist and are widely used in large codebases. Here are some examples:

- The `llvm::function_ref` [2] class template is used throughout LLVM. A quick GitHub search on the LLVM organization reports hundreds of usages both in `llvm` and `clang` [3].
- Facebook's Folly libraries [4] provide a `folly::FunctionRef` [5] class template. A GitHub search shows that it's used in projects `proxygen` and `fbthrift` [6].
- GNU's popular debugger, `gdb` [7], uses `gdb::function_view` [8] throughout its code base. The documentation in the linked header file [9] is particularly well-written and greatly motivates the need for this facility.

Additionally, combining results from GitHub searches *(excluding "`llvm`" and "`folly`")* for "`function_ref`" [10], "`function_view`" [11], "`FunctionRef`" [12], and "`FunctionView`" [13] roughly shows more than 2800 occurrences.

Possible issues

Accepting temporaries in `function_ref`'s constructor is extremely useful in the most common use case: using it as a function parameter. E.g.

```
void foo(function_ref<void()>);

int main()
{
    foo([]{ });
}
```

(on wandbox.org) (https://wandbox.org/permlink/BPtbPeQtErPGj4X7)

The usage shown above is completely safe: the temporary closure generated by the lambda expression is guarantee to live for the entirety of the call to `foo`. Unfortunately, this also means that the following code snippet will result in *undefined behavior*:

```
int main()
{
    function_ref<void()> f{[]{ }};
    // ...
    f(); // undefined behavior
}
```

(on wandbox.org) (https://wandbox.org/permlink/cQPEX2sKjCQjgIki)

The above closure is a temporary whose lifetime ends after the `function_ref` constructor call. The `function_ref` will store an address to a "dead" closure - invoking it will produce undefined behavior [14]. As an example, `AddressSanitizer` detects an invalid memory access in this gist [15]. Note that this problem is not unique to `function_ref`: the recently standardized `std::string_view` [16] has the same problem [17].

I strongly believe that accepting temporaries is a "necessary evil" for both `function_ref` and `std::string_view`, as it enables countless valid use cases. The problem of dangling references has been always present in the language - a more general solution like Herb Sutter and Neil Macintosh's lifetime tracking [18] would prevent mistakes without limiting the usefulness of view/reference classes.

Bikeshedding

The name `function_ref` is subject to bikeshedding. Here are some other potential names:

- `function_view`
- `callable_ref`
- `callable_view`
- `invocable_ref`
- `invocable_view`

- `fn_view`
- `fn_ref`

Acknowledgments

Thanks to **Agustín Bergé**, **Dietmar Kühl**, **Eric Niebler**, **Tim van Deurzen**, and **Alisdair Meredith** for providing very valuable feedback on earlier drafts of this proposal.

Annex: previously open questions

- Why does `operator()` take `Args...` and not `Args&&...`?
  - While taking `Args&&...` would minimize the amount of copies/moves, it would be a pessimization for small value types. Also, taking `Args...` is consistent with how `std::function` works.
- `function_ref<Signature>`'s signature currently only accepts any combination of `const` and `noexcept`. Should this be extended to include *ref-qualifiers*? This would mean that `function_ref::operator()` would first cast the referenced callable to either an *lvalue reference* or *rvalue reference* (depending on `Signature`'s ref qualifiers) before invoking it. See P0045R1 [19] and N4159 [20]) for additional context.
  - LEWG agreed that `const` and `noexcept` have useful cases, but we could not find enough motivation to include support for *ref-qualified* signatures. Nevertheless, this could be added as a non-breaking extension to `function_ref` in the future.
- Constructing a `std::function<Signature>` from a `function_ref<Signature>` is completely different from constructing a `std::string` from a `std::string_view`: the latter does actually create a copy while the former remains a reference. It may be reasonable to prevent implicit conversions from `function_ref` to `std::function` in order to avoid surprising dangerous behavior.
  - LEWG decided to not prevent `std::function` construction from `std::function_ref` as it would special-case `std::function` and there are other utilities in the Standard Library (and outside of it) that would need a similar change (e.g. `std::bind`).
- `function_ref::operator()` is not currently marked as `constexpr` due to implementation issues. I could not figure a way to implement a `constexpr`-friendly `operator()`. Is there any possibility it could be marked as `constexpr` to increase the usefulness of `function_ref`?

    - We agreed that there is probably no way of currently having a `constexpr` `function_ref::operator()` and that we do not want to impose that burden on implementations.

- Should the `!f` precondition when constructing `function_ref` from an instance `f` of `std::function` be removed? The behavior in that case is well-defined, as `f` is guarateed to throw on invocation.

    - LEWG decided to remove the precondition as invoking a default-constructed instance of `std::function` is well-defined.

- The `std::is_nothrow_invocable` constraint in `function_ref` construction/assignment for `noexcept` signatures prevents users from providing a non-`noexcept` function, even if they know that it cannot ever throw (e.g. C functions). Should this constraint be removed? Should an `explicit` constructor without the constraint be provided?

    - LEWG agreed that the constraint should be kept and no extra constructors should be added as users can use a `noexcept` lambda to achieve the same result.

- Propagating `const` to `function_ref::operator()` doesn't make sense when looking at `function_ref` as a simple "reference" class. `const` instances of `function_ref` should be able to invoke a `mutable` lambda, as the state of `function_ref` itself doesn't change. E.g.

    ```
    auto l0 = []() mutable { };
    const function_ref<void()> fr{l0};

    fr(); // Currently a compilation error
    ```

    An alternative is to only propagate `noexcept` from the signature to `function_ref::operator()`, and unconditionally `const`-qualify `function_ref::operator()`. Do we want this?

    - LEWG agreed to mark `function_ref::operator()` `const`, unconditionally.

- We want to avoid double indirection when a `function_ref` instance is initialized with a `reference_wrapper`. `function_ref` could just copy the pointer stored inside the `reference_wrapper` instead of pointing to the wrapper itself. This cannot be covered by the *as-if* rule as it changes program semantics. E.g.

    ```
    auto l0 = []{ };
    auto l1 = []{ };
    auto rw = std::ref(l0);

    function_ref<void()> fr{rw};
    fr(); // Invokes `l0`

    rw = l1;
    fr(); // What is invoked?
    ```

Is adding wording to handle `std::reference_wrapper` as a special case desirable?

  - LEWG decided that special-casing `std::reference_wrapper` is undesirable.

- Is it possible and desirable to remove `function_ref`'s template assignment operator from `F&&` and rely on an implicit conversion to `function_ref` + the default copy assignment operator?

  - LEWG deferred this question to LWG.

- Should `function_ref` only store a `void*` pointer for the callable object, or a `union`? In the first case, seemingly innocent usages will result in undefined behavior:

```
void foo();
function_ref<void()> f{&foo};
f(); // Undefined behavior
```

```
struct foo { void bar(); }
function_ref<void(foo)> f{&foo::bar};
f(foo{}); // Undefined behavior
```

  If a `union` is stored instead, the first usage could be well-formed without any extra overhead (assuming `sizeof(void*) == sizeof(void(*)())`). The second usage could also be made well-formed, but with size overhead as `sizeof(void(C::*)()) > sizeof(void*)`.

  Regardless, the exposition-only members should clearly illustrate the outcome of this decision.

  Note that if we want the following to compile and be well-defined, a `void(*)()` would have to be stored inside `function_ref`:

```
void foo();
function_ref<void()> f{foo};
f();
```

  - LEWG agreed that `function_ref` should fully support the `Callable` concept.

- Should the `function_ref(F&&)` deduction guide take its argument by value instead? This could simplify the wording.

  - LEWG deferred this question to LWG.

References

1. https://vittorioromeo.info/index/blog/passing_functions_to_functions.html#benchmark---generated-assembly (https://vittorioromeo.info/index/blog/passing_functions_to_functions.html#benchmark---generated-assembly)↩
2. http://llvm.org/doxygen/classllvm_1_1function__ref_3_01Ret_07Params_8_8_8_08_4.html (http://llvm.org/doxygen/classllvm_1_1function__ref_3_01Ret_07Params_8_8_8_08_4.html)↩
3. https://github.com/search?q=org%3Allvm-mirror+function_ref&type=Code (https://github.com/search?q=org%3Allvm-mirror+function_ref&type=Code)↩
4. https://github.com/facebook/folly (https://github.com/facebook/folly)↩
5. https://github.com/facebook/folly/blob/master/folly/Function.h#L743-L824 (https://github.com/facebook/folly/blob/master/folly/Function.h#L743-L824)↩
6. https://github.com/search?q=org%3Afacebook+FunctionRef&type=Code (https://github.com/search?q=org%3Afacebook+FunctionRef&type=Code)↩
7. https://www.gnu.org/software/gdb/ (https://www.gnu.org/software/gdb/)↩
8. https://sourceware.org/git/gitweb.cgi?p=binutils-gdb.git;a=blob;f=gdb/common/function-view.h (https://sourceware.org/git/gitweb.cgi?p=binutils-gdb.git;a=blob;f=gdb/common/function-view.h)↩
9. https://sourceware.org/git/gitweb.cgi?p=binutils-gdb.git;a=blob;f=gdb/common/function-view.h (https://sourceware.org/git/gitweb.cgi?p=binutils-gdb.git;a=blob;f=gdb/common/function-view.h)↩
10. https://github.com/search?utf8=%E2%9C%93&q=function_ref+AND+NOT+llvm+AND+NOT+folly+language%3AC%2B%2B&type=Code (https://github.com/search?utf8=%E2%9C%93&q=function_ref+AND+NOT+llvm+AND+NOT+folly+language%3AC%2B%2B&type=Code)↩
11. https://github.com/search?utf8=%E2%9C%93&q=function_view+AND+NOT+llvm+AND+NOT+folly+language%3AC%2B%2B&type=Code (https://github.com/search?utf8=%E2%9C%93&q=function_view+AND+NOT+llvm+AND+NOT+folly+language%3AC%2B%2B&type=Code)↩
12. https://github.com/search?utf8=%E2%9C%93&q=functionref+AND+NOT+llvm+AND+NOT+folly+language%3AC%2B%2B&type=Code (https://github.com/search?utf8=%E2%9C%93&q=functionref+AND+NOT+llvm+AND+NOT+folly+language%3AC%2B%2B&type=Code)↩

13. https://github.com/search?utf8=%E2%9C%93&q=functionview+AND+NOT+llvm+AND+NOT+folly+language%3AC%2B%2B&type=Code (https://github.com/search?utf8=%E2%9C%93&q=functionview+AND+NOT+llvm+AND+NOT+folly+language%3AC%2B%2B&type=Code)↩

14. http://foonathan.net/blog/2017/01/20/function-ref-implementation.html (http://foonathan.net/blog/2017/01/20/function-ref-implementation.html)↩

15. https://gist.github.com/SuperV1234/a41eb1c825bfbb43f595b13bd4ea99c3 (https://gist.github.com/SuperV1234/a41eb1c825bfbb43f595b13bd4ea99c3)↩

16. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3762.html (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3762.html)↩

17. http://foonathan.net/blog/2017/03/22/string_view-temporary.html (http://foonathan.net/blog/2017/03/22/string_view-temporary.html)↩

18. https://github.com/isocpp/CppCoreGuidelines/blob/master/docs/Lifetimes%20I%20and%20II%20-%20v0.9.1.pdf (https://github.com/isocpp/CppCoreGuidelines/blob/master/docs/Lifetimes%20I%20and%20II%20-%20v0.9.1.pdf)↩

19. http://wg21.link/p0045r1 (http://wg21.link/p0045r1)↩

20. http://wg21.link/N4159 (http://wg21.link/N4159)↩